

DISTRIBUTED SYSTEMS

Complete Semester Notes

10 Mark Questions — Detailed Answers with Diagrams

Subject

Distributed Systems (CS)

Type

Semester Exam Prep

Coverage

17 Major Topics

Format

Detail + Diagrams

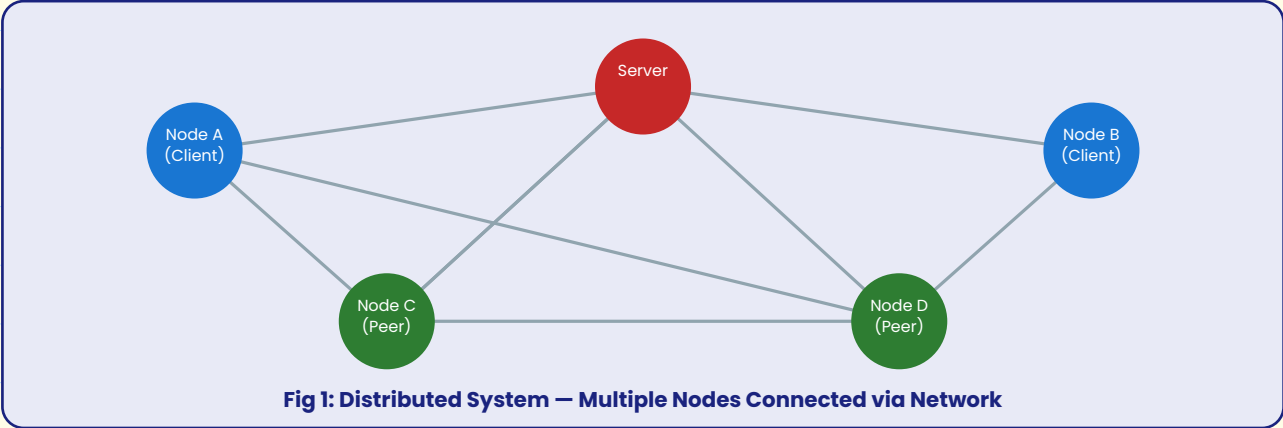
Prepared in Human Handwriting Style — Notebook Format

TABLE OF CONTENTS

1. Distributed Systems — Definition, Types, Models & Limitations
2. File System in Distributed Systems
3. Domain Name System (DNS)
4. Distributed Mutual Exclusion
5. Marshalling in Distributed Systems
6. Name Services — Goals, Implementation & Directory Services
7. Physical Time vs Logical Time
8. Client-Server Communication in DS
9. Key Definitions — Event, Notification, Process, Thread, OS, Client-Server
10. Synchronization in Distributed Systems
11. Clocks in Distributed Systems
12. RPC Protocols
13. CORBA, RMI & Its Services
14. Comparison: DNS vs GNS
15. Global State & Logical Execution
16. Authentication Framework in DS
17. Remote Invocation — Concept
18. IPC — Inter-Process Communication

Q1. Define Distributed System. Explain its Types, Models, and Limitations.

[10 Marks]



Definition

A Distributed System is a collection of independent computers (nodes) that appears to its users as a single coherent system. These computers communicate and coordinate their actions by passing messages over a network. Each node has its own memory and operates independently, but they work together to achieve a common goal.

"A distributed system is one in which the failure of a computer you did not even know existed can render your own computer unusable." – Leslie Lamport

Types of Distributed Systems

Type	Description	Example
Client-Server	Clients request services; servers provide them. Centralized authority and resources.	Web servers, email servers
Peer-to-Peer (P2P)	Each node acts as both client and server. No central authority.	BitTorrent, Blockchain
Cluster Computing	Group of tightly coupled computers working as one. High speed network.	High performance computing (HPC)
Grid Computing	Loosely coupled, geographically distributed machines. Shared resources across organizations.	SETI@home, CERN
Cloud Computing	Resources (compute, storage) provided as services over the Internet. Pay-as-you-go model.	AWS, Azure, Google Cloud
Mobile DS	Includes mobile devices. Deals with mobility, limited resources, intermittent communication.	Mobile apps, IoT

Models of Distributed Systems

Distributed system models describe how components interact and what assumptions are made about the environment:

- Architectural Model: Describes the roles of components (clients, servers, peers) and how they communicate. Includes client-server, multi-tier, peer-to-peer architectures.
- Interaction Model: Describes timing and ordering of interactions. Two sub-models:

□ Synchronous Model — Known bounds on process execution speed, message delivery time, and clock drift.

□ Asynchronous Model — No bounds. Reflects the internet more realistically. Harder to design algorithms for.

- Failure Model: Defines how and when faults can occur. Types: crash failure, omission failure, timing failure, arbitrary (Byzantine) failure.
- Security Model: Defines threats and protective measures. Considers channel security, process security.

Limitations / Challenges of Distributed Systems

Challenge	Explanation
Network Failures	Packets can be lost, delayed, or duplicated. System must tolerate partial failures.
Concurrency	Multiple processes run simultaneously. Synchronization is hard without shared memory.
Lack of Global Clock	No single clock source. Processes have slightly different times (clock skew/drift).
No Shared Memory	Processes can only communicate via message passing, making coordination complex.
Security	Messages travel over networks and can be intercepted. Authentication and encryption needed.
Heterogeneity	Different hardware, OSes, programming languages. Middleware needed for interoperability.
Scalability	System must work as nodes increase from 10 to 10,000+. Design is non-trivial.
Transparency	Users should not see that the system is distributed (location, migration, failure transparency).

Note: A system is only as reliable as its weakest link. In DS, that weakest link is often the network.

Q2. What is a Distributed File System? Explain with its features and architecture.

[10 Marks]

Definition

A Distributed File System (DFS) is a file system that allows files to be stored on multiple servers across a network, while appearing to users as if they are stored locally on a single machine. It provides a transparent, unified view of files regardless of their physical location.

Key Features of DFS

- **Transparency:** Users access files without knowing their physical location. Location transparency, migration transparency, replication transparency are all maintained.
- **Replication:** Files are replicated on multiple servers for fault tolerance and performance. If one server fails, another copy is available.
- **Consistency:** All users see the same version of a file at any time. Write operations are propagated to all replicas.
- **Fault Tolerance:** System continues to operate even if some servers fail. Achieved through replication and redundancy.
- **Scalability:** Can accommodate more servers and users as demand grows without major redesign.
- **Performance:** Caching of files at client side reduces network load and speeds up access.

DFS Architecture

A typical DFS has three main layers:

Layer	Component	Role
Client Layer	Client Module (DFS Client)	Intercepts file requests, translates to RPC calls to server. Handles caching.
Communication Layer	RPC / Message Passing	Transfers requests and responses between client and file server over the network.
Server Layer	File Server (one or more)	Stores files, handles read/write requests, manages replication and metadata.
	Metadata Server (Namespace server)	Tracks file locations, directory hierarchy, and file attributes like permissions.

Examples of DFS

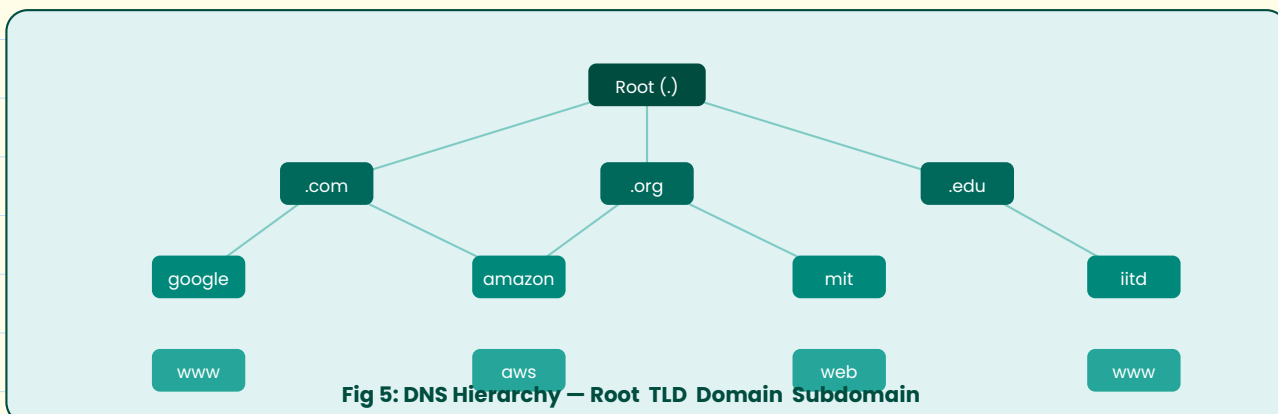
- **NFS (Network File System):** Developed by Sun Microsystems. Standard Unix DFS. Files accessed via RPC.
- **AFS (Andrew File System):** Developed at Carnegie Mellon. Heavy client-side caching. Better scalability than NFS.

- HDFS (Hadoop Distributed File System): Designed for big data. Stores very large files across many nodes with high fault tolerance.
- Google File System (GFS): Designed for Google's internal workloads. Chunk-based storage with one master and multiple chunk servers.

Note: DFS hides distribution. Users see one big file system even though data may be on 100s of servers worldwide.

Q3. Explain the Domain Name System (DNS) in Distributed Systems.

[10 Marks]



What is DNS?

The Domain Name System (DNS) is a hierarchical, distributed naming system that translates human-readable domain names (like `www.google.com`) into machine-readable IP addresses (like `142.250.180.4`). It is often called the "Phone Book of the Internet." DNS is itself a distributed database spread across millions of servers worldwide.

How DNS Works — Step by Step

Step 1: User types `www.example.com` in browser. Browser checks local cache first.

Step 2: If not cached, OS queries the Recursive Resolver (usually provided by ISP or Google `8.8.8.8`).

Step 3: Recursive resolver queries Root Name Server → gets address of `.com` TLD server.

Step 4: Recursive resolver queries `.com` TLD Server → gets address of `example.com`'s authoritative server.

Step 5: Recursive resolver queries Authoritative Name Server → gets IP address of `www.example.com`.

Step 6: IP address returned to browser. Browser connects to the web server at that IP.

DNS Record Types

Record Type	Full Name	Purpose
A	Address Record	Maps hostname to IPv4 address
AAAA	IPv6 Address	Maps hostname to IPv6 address
CNAME	Canonical Name	Alias for another domain name
MX	Mail Exchange	Specifies mail server for domain
NS	Name Server	Identifies authoritative DNS server
TXT	Text Record	Holds arbitrary text (used for SPF, DKIM)

PTR	Pointer Record	Reverse lookup — IP to hostname
------------	----------------	---------------------------------

Note: DNS uses UDP port 53 for queries and TCP port 53 for zone transfers. TTL (Time To Live) controls how long a record is cached.

Q4. Explain Distributed Mutual Exclusion. Discuss its algorithms.

[10 Marks]

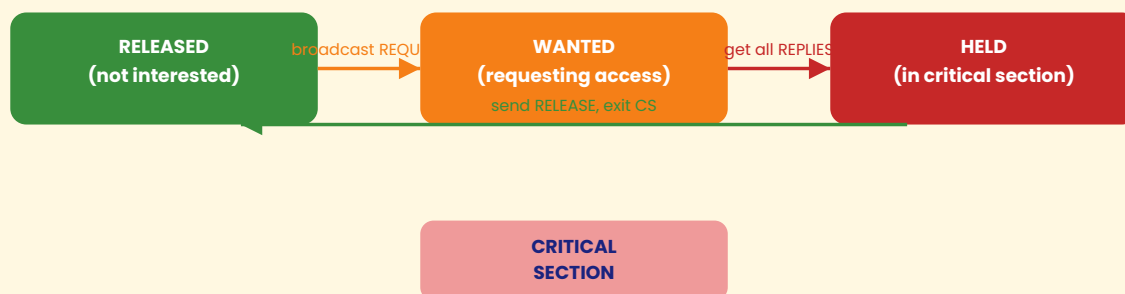


Fig 6: Distributed Mutual Exclusion — State Machine

Definition

Distributed Mutual Exclusion ensures that only one process at a time can enter the Critical Section (CS) in a distributed system, where there is no shared memory. Unlike in traditional systems, we cannot use shared variables or hardware locks.

Requirements for Distributed ME

- Safety (Mutual Exclusion): At most one process can be in the CS at any time.
- Liveness (Progress): A process that wants to enter CS will eventually be able to do so (no deadlock, no starvation).
- Fairness (Ordering): Requests are granted in some fair order (often the order they were made).

Algorithms for Distributed ME

1. Centralized Algorithm (Token-based)

One coordinator process manages access to CS. Process requesting CS sends REQUEST to coordinator. Coordinator maintains a queue. If CS is free, grants REPLY. When done, process sends RELEASE.

- Pros: Simple, fair, low message complexity (3 messages per CS entry).
- Cons: Single point of failure — if coordinator crashes, entire system blocks.

2. Ricart-Agrawala Algorithm (Timestamp-based)

No central coordinator. Process wanting CS broadcasts REQUEST with timestamp to all others. Each other process either replies immediately (if not in CS) or queues the request (if in CS or wanting CS with lower timestamp). Process enters CS after receiving REPLY from all others.

- Pros: No single point of failure. Fair — uses Lamport timestamps for ordering.
- Cons: High message complexity — $2(N-1)$ messages per CS entry for N processes.

3. Token Ring Algorithm

Processes arranged in a logical ring. A single TOKEN circulates the ring. Process can enter CS only when it holds the token. After CS (or if not needed), passes token to next process.

- Pros: Simple, token ensures only one process in CS.
- Cons: If token is lost (process crash), need token regeneration protocol. High latency if CS rarely needed.

Note: In practice, token-ring is popular for simplicity. Ricart-Agrawala is preferred when no central coordinator is acceptable.

Q5. What is Marshalling? Why is it needed in Distributed Systems?

[10 Marks]

Definition

Marshalling (also called Serialization) is the process of converting a data structure or object in memory into a format that can be stored or transmitted over a network, and then reconstructed (unmarshalled/deserialized) at the receiver's end.

Why is Marshalling Needed?

In a distributed system, processes running on different machines communicate by passing messages. These machines may have different:

- Byte Order (Endianness): Intel machines use little-endian; some servers use big-endian. The integer 0x01020304 is stored differently on each.
- Data Representation: Different languages and OSes represent characters, floats, and strings differently.
- Pointer Semantics: Pointers are memory addresses. They are meaningless on another machine's address space.
- Language Differences: Java object, Python dict, C struct — all incompatible natively. Marshalling provides a common format.

Marshalling Process — Step by Step

Sender Side (Marshalling)	Receiver Side (Unmarshalling)
1. Identify data to be sent (int, string, object, struct)	1. Receive the byte stream from network
2. Convert each value to a standard byte format (e.g., big-endian, little-endian)	2. Parse the byte stream according to agreed-upon format
3. Flatten nested structures (arrays, objects) into a linear byte sequence	3. Reconstruct data structures in local memory representation
4. Append metadata: data types, sizes, field names if needed	4. Convert byte order if receiver uses different endianness
5. Send the byte stream over the network	5. Data is ready to use in local program

Marshalling Formats and Standards

- XDR (External Data Representation): Used in RPC/NFS. Converts data to big-endian binary format.
- JSON (JavaScript Object Notation): Human-readable text format. Used in REST APIs. Language-independent.
- XML (Extensible Markup Language): Verbose but highly flexible. Used in SOAP web services.
- Protocol Buffers (Protobuf): Google's binary format. Compact, fast, schema-defined. Used in gRPC.

- Java Serialization: Java-specific mechanism. Objects converted to byte streams. Not cross-language.

Note: Marshalling is also called "Pickling" in Python and "Serialization" in Java/C#. The reverse is "Unpickling/Deserialization/Unmarshalling".

Q6. Explain Name Services — Goals, Implementation and Directory Services.**[10 Marks]****What is a Name Service?**

A Name Service is a distributed system service that provides a mapping between names and attributes (such as addresses, access rights, or other properties) of resources. It allows resources to be accessed by human-friendly names rather than numeric addresses.

Goals of a Name Service

- **User Convenience:** Allow users to refer to resources by meaningful names rather than cryptic addresses.
- **Location Transparency:** The name of a resource does not reveal its physical location. Resources can be moved without changing their names.
- **Scalability:** Must handle millions of name lookups efficiently across a large distributed system.
- **Availability:** Must be highly available. If name service is down, nothing in the system can be found.
- **Consistency:** All clients should see the same name-to-address mapping at any given time.
- **Decentralization:** No single entity should control all names. Administrative authority should be distributable.

How to Implement a Name Service

There are two main approaches:

1. Flat Naming

Names are unstructured strings. A broadcast or flooding mechanism is used to find resources. Works only for small systems. Example: ARP (Address Resolution Protocol) in local networks.

2. Hierarchical Naming

Names are structured in a tree (hierarchy). Different parts of the name tree are managed by different name servers. This approach is scalable and used in DNS. Example: /home/user/docs/file.txt (Unix), www.google.com (DNS).

3. Attribute-based Naming (Directory Services)

Resources have attributes (key-value pairs). Clients search by specifying desired attribute values rather than exact names. Example: LDAP — search for a user with name="Alice" and department="CS".

Directory Services

Directory Service = Name Service + Attribute Storage + Search. It not only maps names to addresses but also stores metadata about resources and allows queries based on attributes.

- LDAP (Lightweight Directory Access Protocol): Standard protocol for directory services. Used in corporate environments for user/resource management. Entries organized in a tree (DIT — Directory Information Tree).
- Active Directory (AD): Microsoft's implementation of LDAP + Kerberos. Manages users, computers, printers in a Windows network.
- DNS: Simplest form of a name service — maps domain names to IPs. Read-only for most users.

Name Resolution

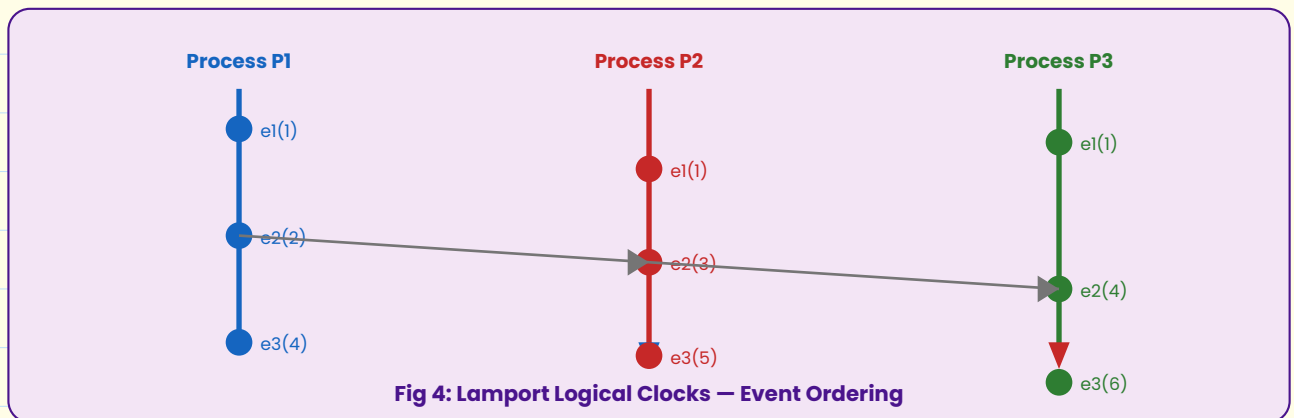
Converting a name to an address is called Name Resolution. Two approaches:

- Iterative Resolution: Client asks one server, which returns address of next server. Client queries each server step by step. Used in DNS (default).
- Recursive Resolution: Client asks one server, which queries the chain on behalf of the client and returns the final answer. Simpler for client, more load on servers.

Note: Name services are fundamental infrastructure. Everything in a DS depends on being able to find resources by name.

Q7. Differentiate between Physical Time and Logical Time in Distributed Systems.

[10 Marks]



Physical Time

Physical time refers to the actual wall-clock time as measured by physical clocks. Each computer has its own hardware clock (oscillator). These clocks run at slightly different rates due to manufacturing differences — this is called Clock Drift.

Problems with Physical Time in DS

- **Clock Skew:** Difference between clock readings of two processes at the same instant. Even a few milliseconds can cause ordering issues.
- **Clock Drift:** Clocks run at slightly different rates. Without synchronization, skew grows over time.
- **No Global Clock:** There is no single authoritative time source that all nodes in a DS can instantly query.

Clock Synchronization Protocols (for Physical Time)

- **NTP (Network Time Protocol):** Synchronizes clocks over internet to within ~1ms accuracy. Hierarchical structure of time servers (Stratum 0, 1, 2...).
- **Cristian's Algorithm:** Client sends request to time server. Server replies with current time. Client accounts for message delay to set its clock.
- **Berkeley Algorithm:** Time daemon asks all machines for time, computes average, sends corrections to each. Good for internal synchronization.

Logical Time

Logical time does not correspond to real time. It is used purely to order events consistently across a distributed system. We often only care about the order of events (did A happen before B?) rather than the exact time they occurred.

Lamport Timestamps (Logical Clocks)

Each process maintains a counter. Rules:

- Before each event, increment counter: $L = L + 1$
- When sending a message, attach current counter value
- When receiving a message: $L = \max(L_{\text{local}}, L_{\text{received}}) + 1$

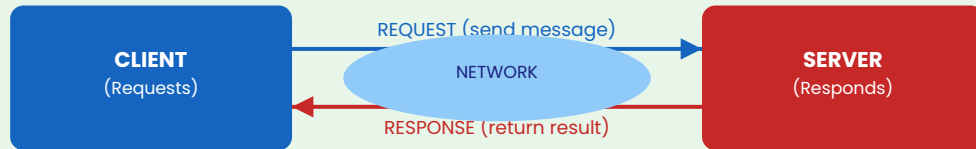
If event A happened before event B (denoted $A \sqsubseteq B$), then $L(A) < L(B)$. However, $L(A) < L(B)$ does NOT necessarily mean $A \sqsubseteq B$ (concurrent events may have ordered timestamps).

Vector Clocks

Each process maintains a vector of counters, one per process. Vector clocks capture causality precisely — if $V(A) < V(B)$, then A causally preceded B. Used in distributed databases, version control (Git uses similar concepts).

Aspect	Physical Time	Logical Time
Based on	Real wall-clock time	Event ordering / causality
Requires sync?	Yes (NTP, Cristian's)	No synchronization needed
Accuracy	Approximate (milliseconds)	Exact causal ordering
Usage	Timeouts, deadlines, leases	Event ordering, debugging, consistency
Clock drift?	Yes — major problem	Not applicable
Example	NTP, GPS time	Lamport clocks, Vector clocks

Note: Lamport's insight: in a distributed system, often we don't need to know WHEN something happened, just WHAT ORDER things happened in.

Q8. How does Client-Server Communication work in Distributed Systems?**[10 Marks]****Fig 2: Client-Server Communication in Distributed System****Overview**

The Client-Server model is the most fundamental paradigm in distributed systems. The client is a process that makes requests for services, and the server is a process that provides those services. Communication happens by exchanging messages over a network.

Communication Patterns

- Request-Reply: Client sends a request message; server processes and sends a reply. Synchronous — client waits. Used in HTTP, RPC.
- One-way (Fire and Forget): Client sends a message but does not wait for a reply. Used in logging, notifications.
- Publish-Subscribe: Clients subscribe to topics. Server (broker) pushes messages to all subscribers when an event occurs.
- Streaming: Server continuously sends data to client (or vice versa). Used in video streaming, sensor data.

Steps in Client-Server Communication (Request-Reply)

Step 1: Client application creates a request message containing: operation name, parameters.

Step 2: Message is marshalled (serialized) into a byte stream for network transmission.

Step 3: Transport layer (TCP/UDP) delivers message across network to server.

Step 4: Server receives message, unmarshals it, and identifies the requested operation.

Step 5: Server executes the operation (e.g., reads database, computes result).

Step 6: Server marshals result into a reply message and sends it back to client.

Step 7: Client receives reply, unmarshals it, and continues execution.

Transport Protocols Used

Protocol	Type	When to Use
TCP	Connection-oriented	When reliability is needed. HTTP, FTP, SMTP.
UDP	Connectionless	When speed matters more than reliability. DNS, video streaming.
HTTP/REST	Application layer	Web services, APIs. Stateless request-reply.
WebSocket	Full-duplex	Real-time bidirectional communication.
gRPC	Binary RPC over HTTP/2	High-performance microservices communication.

Note: The client-server model can be extended to multi-tier. 3-tier = Client Application Server Database Server.

Q9. Define: Event, Notification, Process, Thread, OS, Client-Server.**[10 Marks]****EVENT**

An event is any significant occurrence in a distributed system that can be detected and recorded. Events are the fundamental unit of computation in a distributed system.

- Types of events: (1) Local events — computation within a process; (2) Send event — process sends a message; (3) Receive event — process receives a message.
- In Lamport's model, the "happened-before" ($\sqsubset$) relation orders events across processes based on causality.
- Events are timestamped (using Lamport or vector clocks) to determine their relative order.

NOTIFICATION

A notification is an asynchronous message sent by a server (or publisher) to a client (or subscriber) to inform it of an event or state change without the client explicitly asking for it.

- Notifications are the core of event-driven architectures and publish-subscribe systems.
- Example: A stock price server notifies all subscribed clients when AAPL price changes.
- Contrast with polling — where client repeatedly asks "anything new?" Notifications are more efficient.

PROCESS

A process is an instance of a program in execution. It is the basic unit of work in an OS. Each process has its own private address space, code, data, and resources (file handles, sockets).

- Processes are isolated — they cannot directly access each other's memory (requires IPC).
- In DS, each node runs one or more processes. Processes on different nodes communicate via messages.
- Process states: Running, Ready (waiting for CPU), Blocked (waiting for I/O or message).

THREAD

A thread is a lightweight unit of execution within a process. Multiple threads share the same address space (memory, file handles) of their parent process, making communication between threads fast.

- Threads have their own Program Counter, Stack, and Register set.
- Advantage: Thread creation is faster than process creation; shared memory makes inter-thread communication cheap.
- Concurrency issue: Shared memory requires synchronization (mutex, semaphores) to avoid race conditions.
- In distributed servers, each client request is often handled by a separate thread (thread pool).

OPERATING SYSTEM (OS)

The Operating System is system software that manages computer hardware and provides services for programs. In DS context, the OS provides the platform on which distributed middleware runs.

- OS manages resources: CPU (scheduling), memory, I/O devices, file systems, network.
- Microkernel OS (like Mach, L4): minimal kernel, services run as user-space servers. Better for DS as services can be distributed.
- OS provides IPC mechanisms: pipes, sockets, shared memory — the foundation for DS communication.

CLIENT-SERVER

The Client-Server paradigm is an architecture where clients (service requesters) and servers (service providers) have distinct roles. The client initiates communication; the server waits and responds.

- Client: usually a user-facing program (browser, mobile app). May be diskless, resource-poor.
- Server: typically always-on, resource-rich, handles multiple simultaneous client requests.
- Can be multi-tier: web client $\square$ app server $\square$ database server.
- Modern systems blend this with microservices — a service can be client to one service and server to another.

Q10. Explain Synchronization in Distributed Systems.**[10 Marks]****What is Synchronization?**

Synchronization in distributed systems refers to the coordination of multiple processes that share resources or need to agree on a common state, all without shared memory. Since processes run independently on different machines, they must exchange messages to coordinate their actions.

Types of Synchronization Problems

- **Mutual Exclusion:** Ensuring only one process accesses a shared resource (critical section) at a time. Algorithms: Centralized, Ricart-Agrawala, Token Ring.
- **Election:** Choosing one process as a coordinator/leader when none exists (or when the current leader fails). Algorithms: Bully Algorithm, Ring Election.
- **Clock Synchronization:** Keeping clocks of all processes close enough for coordination. Protocols: NTP, Cristian's, Berkeley.
- **Barrier Synchronization:** All processes must reach a certain point (barrier) before any can proceed. Used in parallel computing.
- **Deadlock Detection/Prevention:** Detecting or preventing circular wait conditions in distributed resource allocation.

Election Algorithms**1. Bully Algorithm**

When a process P notices coordinator is down, it sends ELECTION message to all processes with higher IDs. If none responds, P becomes coordinator (P wins, it "bullies" others). If a process with higher ID responds, it takes over the election.

- Guarantees the process with highest ID becomes coordinator.
- Complexity: $O(N^2)$ messages in worst case.

2. Ring Election Algorithm

Processes arranged in a logical ring. When election needed, process sends ELECTION message with its ID around the ring. Each process compares ID in message with its own, forwards the larger. When originator gets its own ID back, it knows it has the highest ID and declares itself coordinator.

- Complexity: $O(2N)$ messages — more efficient than Bully.

Deadlock in Distributed Systems

A deadlock occurs when a set of processes are each waiting for a resource held by another process in the set, creating a circular wait. In DS, this is harder to detect because there is no global view.

- **Detection:** Maintain a Wait-for Graph (WFG). A cycle in WFG indicates deadlock. Edges: Process P $\rightarrow$ Process Q means P is waiting for a resource held by Q.

- Prevention: Wound-Wait or Wait-Die schemes. Older process kills younger (or waits) to prevent circular dependency.
- Recovery: Terminate one of the deadlocked processes (the "victim") to break the cycle.

Note: Synchronization is the hardest part of distributed systems. Most bugs in DS are synchronization bugs.

Q11. Explain Types of Clocks in Distributed Systems.

[10 Marks]

Physical Clocks

Physical clocks are based on real time, measured by oscillating quartz crystals inside computers. The OS reads the hardware clock register (TSC – Time Stamp Counter) to get current time.

- Clock Drift: Quartz crystals oscillate at slightly different rates. Even 1ms drift per day causes issues in DS.
- Clock Skew: At any instant, two clocks may show different values. Skew = difference between two clock readings at the same moment.

Clock Synchronization Algorithms

Algorithm	Type	Description	Accuracy
Cristian's	External	Client asks time server. Adjusts for half-RTT delay. $C = T_{server} - (RTT/2)$	±40ms
Berkeley	Internal	Daemon polls all nodes, computes average, sends corrections. No external reference.	±1ms
NTP	External	Hierarchical. Stratum 0 = atomic clock. Accounts for variable delays using multiple samples.	±1ms
GPS	External	Satellites provide atomic clock time. Requires GPS hardware receiver on each node.	±1µs
PTP (IEEE 1588)	External	Precision Time Protocol for LAN environments. Hardware timestamping.	±1µs

Logical Clocks — Lamport Timestamps

Proposed by Leslie Lamport in 1978. Tracks causality rather than real time. Each process has a counter C. Rules:

- On any local event: $C = C + 1$
- On send event: attach timestamp C with message
- On receive event: $C = \max(C_{local}, C_{message}) + 1$

Property: If $A \sqsubset B$ (A happens before B), then $LC(A) < LC(B)$. The converse is NOT always true — concurrent events may have ordered Lamport timestamps.

Vector Clocks

Vector clocks solve the limitation of Lamport clocks. Each process P_i maintains a vector $V[1..N]$ where N = number of processes.

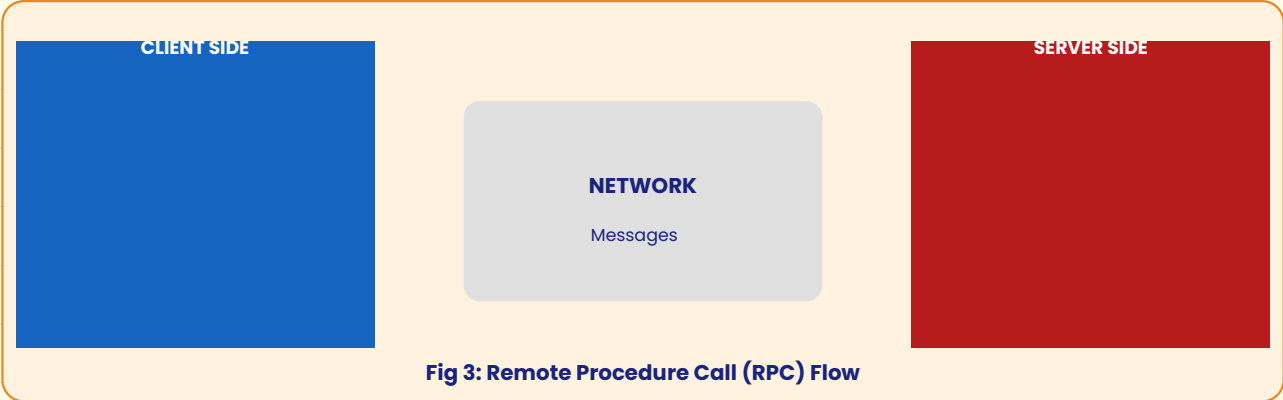
- $V[i]$ = count of events at process P_i
- On local event: $V[i] = V[i] + 1$
- On send: attach entire vector V with message
- On receive: $V[j] = \max(V[j], V_{message}[j])$ for all j; then $V[i]++$

Property: $V(A) < V(B)$ if and only if A causally preceded B. Allows detection of concurrent events. Used in Amazon Dynamo, Riak.

Note: NTP achieves 1-50ms accuracy. Lamport clocks achieve PERFECT causal ordering but tell us nothing about real time.

Q12. Explain RPC Protocols in Detail.

[10 Marks]



What is RPC?

Remote Procedure Call (RPC) is a communication protocol that allows a program to call a procedure (function) located on a remote machine as if it were a local call. The programmer does not need to write explicit network communication code – the RPC framework handles all the low-level details.

How RPC Works

- Client Stub: A proxy that looks like the remote function to the client. It marshals parameters into a message and sends it to the server.
- Server Stub (Skeleton): Receives the message, unmarshals parameters, and calls the actual function. Marshals return value back to client.
- RPCRuntime: Handles low-level communication – sending, receiving, retransmission, acknowledgements.

RPC Semantics (What happens on failure?)

Semantic	Guarantee	Use Case
Maybe RPC	Request sent, no guarantee of execution. No retries.	Non-critical operations
At-least-once	Request retried until acknowledged. May execute multiple times.	Idempotent operations (read, GET)
At-most-once	Request never executed more than once. Server tracks duplicate requests.	Non-idempotent (disk transfer)
Exactly-once	Guaranteed to execute exactly once. Very hard to achieve.	Critical financial transactions

RPC Frameworks

- ONC RPC (Sun RPC): Original RPC framework by Sun Microsystems. Uses XDR for marshalling. Foundation of NFS.
- DCE RPC (Microsoft): Distributed Computing Environment RPC. Basis for DCOM and Windows remote calls.

- gRPC (Google): Modern, high-performance RPC. Uses Protocol Buffers (Protobuf) for serialization. Runs over HTTP/2. Supports streaming.
- JSON-RPC: Lightweight RPC protocol. Uses JSON for encoding. Transport-agnostic (HTTP, WebSocket).
- XML-RPC: Uses XML for encoding. Predecessor to SOAP web services.

Advantages and Disadvantages of RPC

- Advantages: Simple programming model (call remote function like local). Hides network complexity. Generates stub code automatically from IDL (Interface Definition Language).
- Disadvantages: Performance overhead from marshalling/unmarshalling. Network latency is hidden from programmer (may surprise when failures occur). Parameter passing restrictions (can't pass pointers).

Note: RPC is the foundation of all modern distributed computing frameworks. CORBA, Java RMI, gRPC are all RPC-based.

Q13. Explain CORBA and Java RMI and their Services.

[10 Marks]

CORBA — Common Object Request Broker Architecture

CORBA is a standard defined by the OMG (Object Management Group) that allows objects written in different programming languages on different platforms to communicate with each other in a distributed system. It is language and platform neutral.

CORBA Architecture

- IDL (Interface Definition Language): Language-neutral interface specification. Clients and servers both compiled from IDL stubs. IDL $\square$ Java stubs, IDL $\square$ C++ stubs, etc.
- ORB (Object Request Broker): The middleware that handles all communication. It finds remote objects, sends requests, delivers replies. The "bus" of CORBA.
- IIOP (Internet Inter-ORB Protocol): Standard protocol for ORBs to communicate over TCP/IP. Ensures interoperability between different vendors' ORBs.
- Object Adapter (POA — Portable Object Adapter): Connects object implementations to ORB. Manages object lifecycle and request dispatching.

CORBA Services

Service	Purpose
Naming Service	Maps object names to object references. Clients find objects by name.
Trading Service	Find objects by their properties/capabilities (attribute-based lookup).
Event Service	Decoupled communication. Publishers and subscribers. Asynchronous notification.
Transaction Service	ACID transactions across distributed objects. Coordinate commit/rollback.
Concurrency Service	Distributed locking — mutual exclusion for shared CORBA objects.
Security Service	Authentication, authorization, encryption for CORBA communications.
Lifecycle Service	Create, copy, move, delete distributed objects.
Persistence Service	Store state of CORBA objects in databases or files.

Java RMI — Remote Method Invocation

Java RMI allows a Java program to invoke methods of an object residing on a different JVM (possibly on a different machine). Unlike CORBA, it is Java-only (not cross-language), but much simpler to use.

- Remote Interface: Java interface that declares methods callable remotely. Extends `java.rmi.Remote`. Each method declares throws `RemoteException`.
- Remote Object Implementation: Server class that implements the remote interface. Extends `UnicastRemoteObject`.

- **RMI Registry:** Simple name service for RMI. Server registers objects. Client does `Naming.lookup("rmi://host/ObjectName")` to get a reference.
- **Stubs and Skeletons:** Generated by `rmic` compiler (or automatically in Java 5+). Stub on client marshals the call; skeleton on server unmarshals and invokes.
- **Java Serialization:** RMI uses Java's built-in serialization to marshal/unmarshal parameters. Objects passed by copy, not by reference.

Feature	CORBA	Java RMI
Language support	Multi-language (Java, C++, Python...)	Java only
Interoperability	High — uses IIOP standard	Limited — JVM to JVM only
Marshalling	IDL-defined, language-neutral	Java serialization
Complexity	Complex — IDL + ORB setup	Simpler — pure Java
Performance	Higher overhead (IIOP)	Better within Java ecosystem
Used in	Banking, telecom, legacy enterprise	Java EE, Spring, academic projects

Note: CORBA dominated enterprise DS in the 1990s–2000s. Today, REST APIs and gRPC have largely replaced both CORBA and RMI.

Q14. Compare DNS (Domain Name System) and GNS (Global Name Service).

[10 Marks]

Overview

DNS was designed primarily for the Internet – mapping hostnames to IP addresses. GNS (Global Name Service, developed at DEC) was designed as a general-purpose distributed name service for enterprise environments, with richer features than DNS.

Feature	DNS	GNS (Global Name Service)
Primary Purpose	Map domain names to IP addresses	General-purpose naming for any resource attribute
Name Structure	Hierarchical, right-to-left (root at right): www.example.com	Hierarchical, left-to-right: /aiken/proj/file
Data Model	Simple – maps name to a few record types (A, NS, etc.)	Rich – stores arbitrary attributes as key-value pairs
Attribute lookup	Not supported – only name lookup	Supports searching by attribute values
Replication	Zone-based – each zone copied to secondary servers	Server-based – individual records can be replicated
Caching	TTL-based caching at resolvers. May serve stale data	Callbacks – server notifies client when data changes
Consistency	Eventual consistency. Updates propagate slowly	Stronger consistency – callback mechanism
Fault Tolerance	Separate primary/secondary name servers per zone	Distributed across multiple name servers with replication
Scale	Global scale – billions of records	Enterprise/campus scale
Lookup type	Forward: name to IP; Reverse: IP to name	Forward and attribute-based lookups
Protocol	UDP/TCP port 53	Custom protocol
Openness	Open standard (IETF RFC)	Proprietary (DEC research project)

Key Differences Summarized

- Purpose: DNS is specific (hostname to IP). GNS is general (any name to any attribute set).
- Data richness: GNS stores richer metadata. DNS stores a limited set of record types.
- Consistency: GNS uses callbacks for notification of changes. DNS relies on TTL expiry (may serve stale data for hours).
- Adoption: DNS is the universal internet standard. GNS remains a research/academic concept; it influenced later directory services like LDAP.

Note: GNS ideas heavily influenced the design of LDAP and Active Directory, which are the dominant enterprise naming/directory services today.

Q15. Define Global State and Logical Execution in Distributed Systems.**[10 Marks]****Global State**

The Global State of a distributed system at any point in time is the combination of the local states of all processes and the messages that are currently in transit (in the channels) between them.

Global State = $\square$ (Local States of all processes) + State of all Communication Channels Formally: $GS = \{S_1, S_2, \dots, S_n, C_{12}, C_{21}, C_{13}, \dots\}$ where S_i = state of process i and C_{ij} = messages in channel from i to j .

Why Global State is Hard to Capture

- There is no global clock — you cannot "pause" all processes simultaneously and read their states.
- Messages are in-flight — a process may have sent a message but the receiver hasn't received it yet.
- The act of observing a process's state requires communication, which itself changes the state.

Consistent Global State (Consistent Cut)

A consistent cut is a "snapshot" of the global state that respects causality. Formally, if event e_2 is in the cut and $e_1 \sqsubseteq e_2$ (e_1 happened before e_2), then e_1 must also be in the cut.

Chandy-Lamport Snapshot Algorithm

The most famous algorithm for capturing a consistent global snapshot:

- Any process can initiate by recording its own state and sending a special MARKER message on all outgoing channels.
- When a process receives a MARKER for the first time: record own state, start recording incoming channel state.
- When MARKER received on a channel that is already being recorded: stop recording that channel — channel state = messages received since own snapshot.
- Snapshot is complete when all processes have received MARKERS on all incoming channels.

Applications of Global State

- Deadlock Detection: Examine global state to find circular waits.
- Termination Detection: Determine if a distributed computation has completed.
- Checkpointing: Save consistent state for recovery after failures (checkpoint and rollback).
- Debugging: Replay execution from a saved consistent state.

Logical Execution

Logical execution (or logical execution history) of a distributed system is the sequence of events in all processes, ordered according to their causal relationships (using happens-before $\sqsubseteq$ relation), rather than by wall-clock time.

- Two events are concurrent if neither causally precedes the other ($A \not\sqsubseteq B$). They can be placed in any order in the logical execution.
- Logical execution is the foundation for reasoning about correctness, consistency, and debugging in distributed programs.
- Tools like Lamport timestamps and vector clocks help reconstruct the logical execution from execution traces.

Note: The Chandy-Lamport algorithm won the Dijkstra Prize — it is one of the most elegant and important results in distributed computing.

Q16. Explain the Authentication Framework in Distributed Systems.

[10 Marks]

Why Authentication in DS?

In a distributed system, services are accessed over a network. A server cannot directly verify the identity of a client process — the network can carry spoofed messages. Authentication is the process of verifying that a user or process is who it claims to be. Without authentication, DS are vulnerable to impersonation attacks.

Authentication Concepts

- Principal: Any entity (user, process, host) that can be authenticated. Identified by a unique name.
- Credential: Proof of identity — password, digital certificate, cryptographic key.
- Authentication Server (AS): Trusted third party that vouches for identities. Issues credentials.
- Trusted Third Party: An entity both client and server trust. Key to many DS auth frameworks.

Kerberos Authentication System

Kerberos is the most widely deployed authentication framework for distributed systems (used in Windows Active Directory, MIT environments). It uses symmetric cryptography and a trusted Key Distribution Center (KDC).

- Step 1:** Client sends username to Authentication Server (AS). AS verifies user exists.
- Step 2:** AS returns Ticket-Granting Ticket (TGT) encrypted with client's key (derived from password). Also returns session key encrypted with client's key.
- Step 3:** Client decrypts using password □ gets TGT and session key. Client never sends password over network.
- Step 4:** To access a service, client sends TGT to Ticket-Granting Server (TGS) with service name.
- Step 5:** TGS issues a Service Ticket encrypted with service's secret key. Returns to client.
- Step 6:** Client presents Service Ticket to target service server. Server decrypts with its own key, verifies identity.
- Step 7:** Mutual authentication: server sends back authenticator so client can also verify server identity.

Other Authentication Methods in DS

Method	How It Works	Strength
Password Auth	Client sends password. Server verifies against stored hash.	Weak — password can be intercepted
Public Key (PKI)	Client signs message with private key. Server verifies with public key.	Strong — based on PKI certificates.

OAuth 2.0	Delegation protocol. User authorizes app to access resources on their behalf.	Strong on the web/mobile APIs
JWT (JSON Web Token)	Stateless token with claims, signed by server. Client presents it to access resources.	Good for stateless REST APIs
SAML	XML-based. Identity provider asserts user identity to service provider.	Enterprise SSO (Single Sign-On)

Note: Kerberos tickets have a limited lifetime (typically 8-10 hours). This reduces the window of vulnerability if a ticket is stolen.

Q17. Explain the Concept of Remote Invocation in Distributed Systems.**[10 Marks]****What is Remote Invocation?**

Remote Invocation is the general concept of invoking (calling) an operation/method on a remote object or service located on a different machine. It extends local procedure calls to the distributed environment. Remote invocation is the foundation of all distributed programming models.

Forms of Remote Invocation**RPC (Remote Procedure Call)**

Oldest and most basic form. Client calls a function on a server as if it were local. Synchronous — client blocks until result returns. Server is stateless (in basic RPC).

Examples: ONC RPC (used in NFS), DCE RPC, gRPC

RMI (Remote Method Invocation)

Object-oriented version of RPC. Client invokes methods on remote objects. Remote objects can be passed as parameters. Java RMI is the most common implementation.

Examples: Java RMI, .NET Remoting

Event-based Communication

Asynchronous form. Publisher generates events; subscribers receive them via a middleware (event broker). No direct connection between publisher and subscriber.

Examples: CORBA Event Service, JMS, Apache Kafka

Message Queuing

Sender deposits messages in a queue; receiver picks up when ready. Decouples sender and receiver in time. Suitable for high-throughput async processing.

Examples: RabbitMQ, AWS SQS, ActiveMQ

RESTful Invocation (HTTP)

Modern form using HTTP verbs (GET, POST, PUT, DELETE). Resources identified by URLs. Stateless, lightweight. Dominates web-based distributed systems.

Examples: REST APIs, Web Services, Microservices

Comparison of Remote Invocation Forms

Property	RPC	RMI	Message Queue	REST
Paradigm	Procedural	Object-oriented	Asynchronous	Resource-based
Coupling	Tight	Tight	Loose	Loose

Synchrony	Synchronous	Synchronous	Asynchronous	Both
Language	Multi-lang	Java-only	Multi-lang	Multi-lang
Complexity	Medium	Low (Java)	Medium	Low

Note: Modern microservices architectures combine REST (for synchronous ops) and Message Queues (for async ops) for the best of both worlds.

Q18. Explain IPC (Inter-Process Communication) in Detail.**[10 Marks]****Fig 7: IPC Mechanisms in Distributed Systems****What is IPC?**

Inter-Process Communication (IPC) refers to the mechanisms that allow processes to communicate and synchronize their actions. In a distributed system, processes on different machines can only communicate via IPC mechanisms over the network. IPC is the backbone of all distributed computing.

IPC Mechanisms**1. Pipes (|)**

Unidirectional communication channel. Data flows in one direction. Process writes to write-end; another reads from read-end. Named pipes (FIFOs) work across unrelated processes.

- Used heavily in Unix shell: `ls | grep .txt`
- Limitation: Unidirectional, only between related processes (anonymous pipes).
- In DS context: used within a single machine, not across network.

2. Sockets

The fundamental IPC mechanism for distributed systems. A socket is an endpoint for communication. Combination of IP address + port number uniquely identifies a socket endpoint globally.

- Stream sockets (TCP): reliable, connection-oriented byte stream.
- Datagram sockets (UDP): unreliable, connectionless message delivery.
- Domain sockets (Unix): for processes on same machine. Faster than TCP loopback.
- Every network application (HTTP, FTP, SSH, RPC) uses sockets at the lowest level.

3. Message Queues

Processes communicate by sending messages to named queues. Receiver reads from queue. Messages are stored until retrieved. Asynchronous — sender and receiver do not need to be running simultaneously.

- System V Message Queues: OS-level IPC. `msgget()`, `msgsnd()`, `msgrcv()`.
- POSIX Message Queues: `mq_open()`, `mq_send()`, `mq_receive()`.

- Distributed MQ: RabbitMQ, Kafka — work across network, not just single machine.

4. Shared Memory

Two or more processes map the same physical memory region into their address spaces. Communication is extremely fast — just memory reads and writes. No kernel involvement after setup.

- Fastest IPC mechanism on a single machine.
- Requires synchronization (semaphores or mutexes) to avoid race conditions.
- NOT available across network — only on same machine. Remote systems use DSM (Distributed Shared Memory) as an abstraction.

5. Semaphores

A semaphore is a synchronization primitive (an integer with two atomic operations: wait/P and signal/V). Used to protect shared resources and coordinate process execution. Binary semaphore = mutex.

- Binary Semaphore: 0 or 1. Used as mutex lock.
- Counting Semaphore: Integer ≥ 0 . Tracks number of available resources.
- Used alongside shared memory for safe concurrent access.

6. Remote Procedure Call (RPC)

As described earlier — the highest-level IPC mechanism for distributed systems. Hides all network communication behind a function call interface.

- Used in NFS (ONC RPC), CORBA (IIOP), gRPC.
- Programmer calls a function — middleware handles all IPC details.

IPC Comparison Table

Mechanism	Location	Speed	Sync Needed?	Best For
Pipes	Same machine	Fast	No	Shell pipelines
Sockets	Across network	Medium	No	Distributed apps
Message Queue	Same/across net	Medium	No	Async processing
Shared Memory	Same machine	Very fast	Yes (semaphore)	High-speed local IPC
Semaphores	Same machine	Very fast	Is sync itself	Access control
RPC	Across network	Slow (network)	No	Distributed function calls

Note: In practice: Use sockets for distributed IPC, shared memory + semaphores for fast local IPC, and RPC/REST for high-level distributed invocation.

QUICK REVISION — Key Points to Remember

Topic	Key Point
Distributed System	Multiple independent computers appearing as one. Connected via network.
Types	Client-Server, P2P, Cluster, Grid, Cloud, Mobile DS.
Limitations	Network failures, no global clock, concurrency, security, heterogeneity.
DFS	Files stored on multiple servers; appears as one system. Examples: NFS, HDFS, GFS.
DNS	Hierarchical naming system. Maps domain names to IPs. Uses UDP port 53.
Distributed ME	One process in CS at a time. Algorithms: Centralized, Ricart-Agrawala, Token Ring.
Marshalling	Serializing data for network transmission. Formats: XDR, JSON, Protobuf.
Name Services	Maps names to resource attributes. Goals: transparency, scalability, availability.
Physical Time	Real wall-clock time. Problem: clock skew and drift. Solution: NTP, Berkeley.
Logical Time	Causal ordering of events. Lamport clocks, Vector clocks.
Client-Server	Client requests → Server responds. Basis of most DS architectures.
Process vs Thread	Process = independent memory space. Thread = lightweight, shares process memory.
Synchronization	Coordinating processes: ME, election, deadlock detection.
RPC	Call remote function as if local. Stubs handle marshalling. Semantics: at-least-once, etc.
CORBA	Multi-language DS framework. ORB + IDL + IIOP. Services: naming, events, transaction.
Java RMI	Java-only remote method invocation. Registry for name lookup. Java serialization.
DNS vs GNS	DNS: internet naming, hostname → IP. GNS: general naming, richer attributes, callbacks.
Global State	All process states + channel states. Captured using Chandy-Lamport snapshot.
Authentication	Kerberos: TGT + Service Tickets. PKI, OAuth, JWT also used.
Remote Invocation	RPC, RMI, Events, Message Queues, REST — all forms of remote invocation.
IPC	Pipes, Sockets, Message Queues, Shared Memory, Semaphores, RPC.

All the best for your exams! Remember: Distributed Systems = Concurrency + Communication + Failure Handling.